

DevSecOps: Integrando Segurança na Entrega Contínua

A cultura DevSecOps garante software resiliente sem comprometer a velocidade do CI/CD.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Sumário

- O Surgimento do DevSecOps
- Princípios de Shift Left e Automação
- Ferramentas SAST, DAST e SCA
- Automação e Métricas de Desempenho
- Desafios e Debate DevSecOps

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



O Surgimento do DevSecOps

DevSecOps é a evolução do DevOps, integrando Desenvolvimento (**Dev**), Segurança (**Sec**) e Operações (**Ops**) de forma simbiótica e contínua. Diferente dos modelos tradicionais, a segurança é **embutida** desde a concepção até a entrega, tornando-se uma responsabilidade compartilhada. O objetivo é garantir software **resiliente** sem comprometer a velocidade da entrega contínua (CI/CD).



Hub de Sincronização: Conectando sistemas para integração segura e eficiente.

Impacto do Atraso

100:1

Custo produção vs. requisitos

6x

Custo implementação vs. design

15x

Custo testes vs. design

A correção de vulnerabilidades em estágios avançados do ciclo de vida de desenvolvimento é drasticamente mais cara e demorada. A proporção estimada do custo para corrigir uma falha de segurança em produção, comparado à fase de requisitos, é de **100:1**. Erros identificados na fase de implementação custam aproximadamente **6 vezes mais** do que na fase de design. Quando chegam à fase de testes, esse custo sobe para **15 vezes** o custo da fase de design. A implementação do DevSecOps, como vimos na introdução, visa mitigar esses riscos financeiros e operacionais através da detecção precoce de falhas de segurança.

DevOps vs. DevSecOps

Esta tabela detalha as principais diferenças entre DevOps tradicional e DevSecOps, enfatizando a integração da segurança em todo o ciclo de vida do desenvolvimento.

Atributo	DevOps Tradicional	DevSecOps
Foco Principal	Velocidade e Agilidade	Velocidade com **Segurança**
Papel da Segurança	Verificação *reativa* ao final do ciclo	Integrada em cada commit e fase
Responsabilidade	Dividida entre Dev e Ops	Compartilhada entre Dev, Sec e Ops
Automação	CI/CD para deploy e testes funcionais	CI/CD incluindo scans de segurança (SAST/DAST)
Feedback	Focado em performance e bugs	Focado em **vulnerabilidades** e riscos

O Conceito Shift Left no DevSecOps

1

Deslocar Segurança para Esquerda

O 'Shift Left' move as práticas de segurança para as fases iniciais do ciclo de desenvolvimento, integrando-as desde o *design* para identificar e corrigir vulnerabilidades precocemente.

2

Modelagem de Ameaças

Realizada na fase de *design*, esta etapa antecipa potenciais riscos de segurança, ajudando a criar um sistema mais robusto desde o início.

3

Análise Estática (SAST)

Ferramentas de SAST executam a análise de código enquanto ele é escrito no IDE, fornecendo feedback imediato sobre vulnerabilidades, como *SQL injection* ou *cross-site scripting* (XSS).

O Conceito Shift Left no DevSecOps

4

Revisão de Código Segura

Durante o processo de *code review* por pares, o foco é expandido para incluir a identificação de falhas de segurança, garantindo que o código esteja em conformidade com as políticas de segurança.

5

Testes Unitários de Segurança

São criados testes específicos para validar requisitos mínimos de proteção em nível de função, verificando a resiliência do código contra ataques conhecidos e configurando a segurança da aplicação.

6

Redução de Vulnerabilidades Críticas

Ao adotar o 'Shift Left', evita-se que vulnerabilidades críticas cheguem às fases finais de *deploy*, reduzindo custos de correção e o risco de incidentes de segurança.

Pilares: Cultura e Automação

Cultura de Segurança

A segurança não é apenas tarefa do 'time de segurança', mas uma mentalidade intrínseca aos desenvolvedores. Envolve **treinamento** e **empoderamento** das equipes, garantindo que todos compreendam seu papel na proteção do ciclo de vida do software.

Automação Total

Ferramentas de segurança devem ser integradas ao *pipeline* de CI/CD. Se o processo for manual, ele se tornará um obstáculo à velocidade do DevOps, comprometendo a **eficiência** e a **agilidade** da entrega de software seguro.

Monitoramento Contínuo

- A segurança não termina no *deploy*. É essencial observar o comportamento do sistema em **tempo real** para
- detectar ataques e anomalias de dia zero, garantindo uma postura de segurança *proativa* e **adaptável** pós-lançamento.
-
-

Pipeline de Segurança Integrado



1

Planejamento

Definição de requisitos de segurança e *modelagem de ameaças* para identificar e mitigar riscos proativamente.

2

Codificação

Uso de *linters* e ferramentas de SAST (*Static Application Security Testing*) integradas ao editor para análise estática do código-fonte.

3

Build

Análise de dependências (SCA - *Software Composition Analysis*) para verificar bibliotecas e componentes de código aberto vulneráveis.

Pipeline de Segurança Integrado

4

Teste

Execução de DAST (*Dynamic Application Security Testing*) em ambientes de *staging* para identificar vulnerabilidades em tempo de execução.

5

Deploy

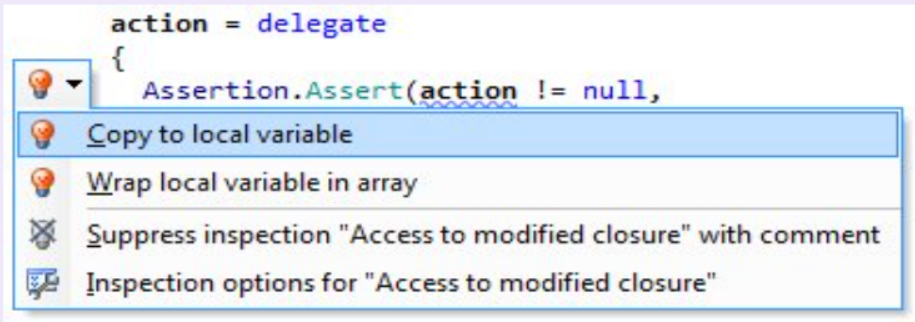
Verificação de configurações de infraestrutura como código (IaC) e segurança dos ambientes de implantação.

6

Operação e Monitoramento

Gestão de logs, SIEM (*Security Information and Event Management*) e resposta a incidentes para detecção e remediação contínuas.

Análise Estática (SAST)



ReSharper sugere melhorias para análise estática de código.

- A **definição** de SAST envolve a inspeção do código-fonte, binários ou *bytecodes* sem a execução da aplicação, buscando vulnerabilidades diretamente no código.
- Entre as **vantagens**, destaca-se a capacidade de identificar falhas como *SQL Injection*, *Cross-Site Scripting* (XSS) e *Buffer Overflows* nas fases iniciais do desenvolvimento, fornecendo feedback imediato.
- Por outro lado, suas **limitações** incluem um potencial alto para falsos positivos e a incapacidade de detectar problemas de configuração de *runtime* ou autenticação.
- Exemplos de **ferramentas** SAST amplamente utilizadas são SonarQube, Checkmarx e Snyk Code, que automatizam essa análise no ciclo de desenvolvimento.



Análise Dinâmica (DAST)



Escudo de cibersegurança protegendo redes e dados digitais.

- **Funcionamento:** O DAST simula ataques externos em uma aplicação em execução para revelar vulnerabilidades de *runtime*, como falhas de sessão ou configuração de servidor.
- **Vantagens:** Destaca-se pelo baixo índice de falsos positivos e por ser independente da linguagem de programação, adaptando-se a diversos ambientes.
- **Limitações:** Exige uma aplicação funcional e, embora identifique vulnerabilidades, não aponta a linha exata do código-fonte problemático.
- **Exemplos:** Ferramentas como OWASP ZAP e Burp Suite são amplamente utilizadas para realizar testes de segurança dinâmicos em aplicações web.

SAST vs. DAST

Esta tabela detalha as principais diferenças entre SAST (Static Application Security Testing) e DAST (Dynamic Application Security Testing).

Característica	SAST	DAST
Ponto de Vista	White-box (Interno)	Black-box (Externo)
Fase	Desenvolvimento / Build	Teste / QA / Produção
Requer Execução?	Não	Sim
Custo de Correção	Menor (precoce)	Maior (tardio)
Deteccção	Erros lógicos e de sintaxe	Erros de configuração e runtime

Dependências e SCA



Inventário de Código

Identificar todas as bibliotecas e dependências utilizadas no projeto. A maioria das aplicações modernas é composta por **80% a 90% de bibliotecas de terceiros**.

Vulnerabilidades Conhecidas

Cruzar as versões de componentes com bancos de dados de CVEs (Common Vulnerabilities and Exposures) para detectar riscos. O *Software Composition Analysis* (SCA) é crucial aqui.

Gestão de Licenças

Garantir a conformidade jurídica das licenças open source e evitar riscos na cadeia de suprimentos de software.

Infraestrutura como Código (IaC)



Redes digitais e seus nós representam a infraestrutura gerenciada por código.

- **Definição IaC:** A infraestrutura é gerenciada e provisionada usando arquivos de código, que são versionados e submetidos a controle como qualquer outra parte da base de código da aplicação.
- **Escaneamento Preventivo:** Ferramentas como checkov e TFSec automatizam a inspeção de arquivos como Terraform, CloudFormation e Kubernetes manifests antes do provisionamento.
- **Vulnerabilidades Comuns:** Identifique configurações inseguras, como grupos de segurança excessivamente permissivos (0.0.0.0/0), ausência de criptografia em bancos de dados ou buckets S3, e credenciais (secrets)

Automação: Gated Check-ins

- A automação atua como um "guardião" da qualidade no pipeline DevSecOps.
- Os **Gated Check-ins** interrompem o processo de integração (Break the Build) sob condições pré-definidas, por exemplo:
- Vulnerabilidades críticas forem detectadas (nível > zero).
- A cobertura de testes de segurança estiver abaixo do limite estabelecido.
- Novas dependências inseguras forem introduzidas.
- Isso impede que códigos vulneráveis avancem, institucionalizando **padrões de segurança**.

Métricas de Desempenho



Dashboard de desempenho: métricas essenciais para análise e otimização.

- **MTTD (Mean Time to Detect):** Mede o tempo médio necessário para identificar uma vulnerabilidade de segurança.
- **MTTR (Mean Time to Repair):** Indica o tempo médio para corrigir uma vulnerabilidade e realizar o *deploy* do *patch* em produção.
- **A Taxa de Falsos Positivos** representa o percentual de alertas gerados pelas ferramentas que são considerados irrelevantes.
- **A Densidade de Vulnerabilidades** quantifica o número de falhas de segurança por KLOC (milhares de linhas de código).

Principais Desafios DevSecOps



Fadiga de Alertas

O excesso de falsos positivos pode levar as equipes a ignorar os *scans* de segurança, minando a eficácia das verificações automatizadas.



Resistência Cultural

A percepção equivocada de que a segurança é um obstáculo que atrasa o desenvolvimento, gerando atritos entre equipes de Dev e Sec.



Falta de Especialistas

Escassez de profissionais com competência técnica em desenvolvimento e segurança simultaneamente (*SecDev*, *DevSecEng*).



Complexidade Técnica

Dificuldade em integrar ferramentas legadas e processos manuais em *pipelines* CI/CD modernos, exigindo reengenharia significativa.



Discuta com o colega ao lado

Como podemos equilibrar a necessidade de entregas rápidas com a rigorosidade dos testes de segurança no pipeline de CI/CD (Integração Contínua/Entrega Contínua)?

Considere os compromissos aceitáveis e os papéis na decisão de interrupção de um deploy.

Considerando os pilares do DevSecOps e o conceito de *Shift Left*, qual das seguintes afirmações descreve corretamente a abordagem de segurança em um ciclo de desenvolvimento ágil?

- A SAST (Static Application Security Testing) é uma técnica de teste de segurança de caixa-preta que requer que a aplicação esteja em execução para analisar seu comportamento.
- B O DevSecOps prioriza auditorias de segurança manuais exclusivamente na fase pós-deploy, concentrando-se na detecção de vulnerabilidades em produção.
- C O conceito de *Shift Left* enfatiza a integração e o início das atividades de segurança o mais cedo possível no ciclo de vida do desenvolvimento de software.
- D O principal objetivo do DevSecOps é estabelecer uma clara separação entre as equipes de segurança e desenvolvimento, designando responsabilidades distintas para cada uma.

Considerando os pilares do DevSecOps e o conceito de *Shift Left*, qual das seguintes afirmações descreve corretamente a abordagem de segurança em um ciclo de desenvolvimento ágil?

- ✗ SAST (Static Application Security Testing) é uma técnica de teste de segurança de caixa-preta que requer que a aplicação esteja em execução para analisar seu comportamento.
- ✗ O DevSecOps prioriza auditorias de segurança manuais exclusivamente na fase pós-deploy, concentrando-se na detecção de vulnerabilidades em produção.
- ✓ O conceito de *Shift Left* enfatiza a integração e o início das atividades de segurança o mais cedo possível no ciclo de vida do desenvolvimento de software.
- ✗ O principal objetivo do DevSecOps é estabelecer uma clara separação entre as equipes de segurança e desenvolvimento, designando responsabilidades distintas para cada uma.

Conclusão

- DevSecOps evolui DevOps, integrando segurança de forma contínua e compartilhada, mitigando riscos financeiros e operacionais em até 100 vezes.
- O conceito Shift Left antecipa a segurança para as fases iniciais do desenvolvimento, como modelagem de ameaças e SAST, reduzindo custos.
- SAST, DAST e SCA são pilares técnicos, inspecionando código estático, dinâmico e dependências, complementando-se na detecção de vulnerabilidades.
- A automação via Gated Check-ins e monitoramento contínuo com métricas como MTTR, são cruciais para a maturidade DevSecOps.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

